

Sprawozdanie z projektu z programowania obiektowego

1 Założenia projektu:

Celem projektu jest utworzenie symulacji robota antropomorficznego z trzema stopniami swobody. Założono układ współrzędnych sferycznych. Cały projekt zrealizowany został zgodnie z założeniami programowania obiektowego.

2 Struktura:

Projekt podzielony jest na 7 plików w pythonie każdy odpowiadający innej czynności robota:

- **main.py** - główny plik obsługujący główną pętlę działania programu
- **autonomous.py** - odpowiadający za autonomiczne sortowanie klocków przy implementacji systemu wizyjnego robota
- **controller.py** - odpowiada za wprowadzanie współrzędnych podczas implementacji zaganień odwrotnej kinematyki, oraz za suwaki
- **enviroment.py** - odpowiada za fizykę środowiska i kostki
- **robot.py** - obsługuje polecenia ruchu robota
- **teach_pendant.py** - moduł pamięci robota, pozwala na implementacje uczenia i pracy
- **vision.py** - system wizyjny robota potrzebny do autonomicznego sortowania

3 Biblioteki:

Do implementacji projektu użyto bibliotek

- **Pybullet** - do obrotów wokół osi i kątów
- **OpenCv** - do implementacji systemu wizyjnego robota
- **Numpy** - do obliczeń matematycznych

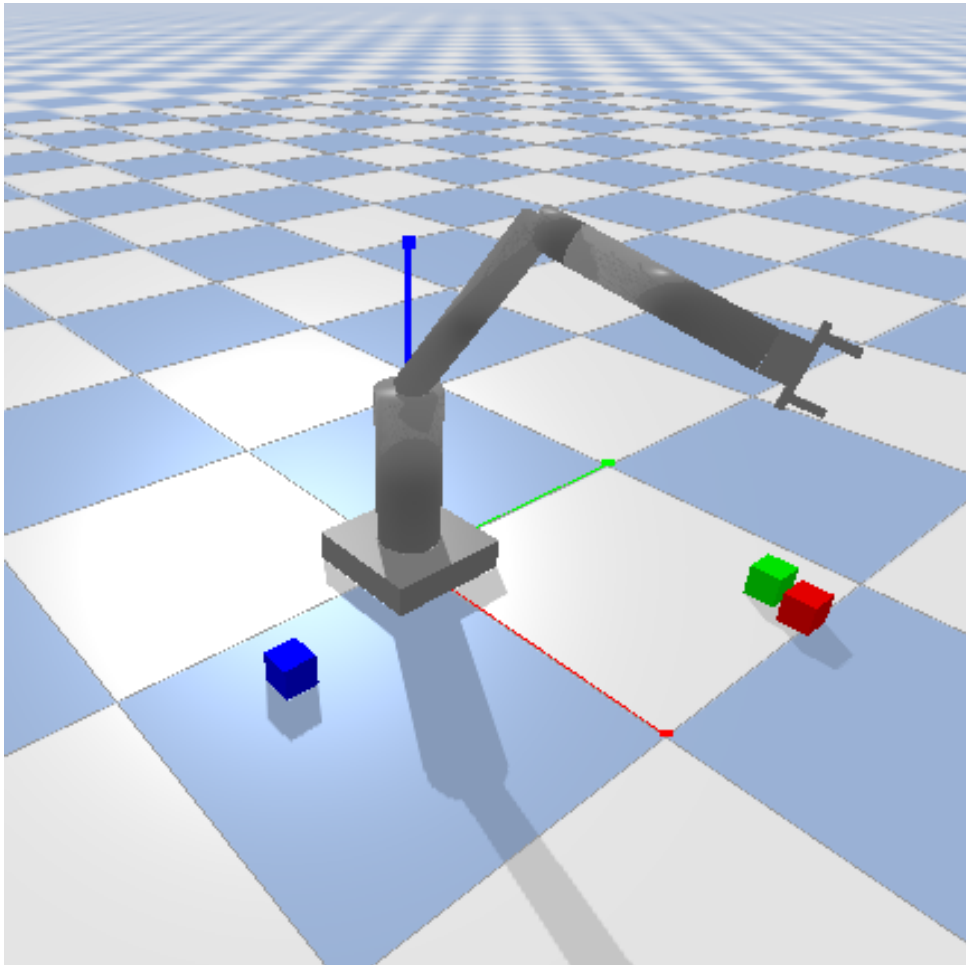
4 Robot:

Model robota został zaprogramowany przy wykorzystaniu plików URDF (Unified Robotic Description Format) do stworzenia modelu robota oraz biblioteki Pybullet do opisów kinematyki robota, w celu ominięcia stosowania macierzy obrotu.

Napisano osobne funkcje odpowiadające za:

- a. Skanowanie stawów robota, i zapisanie tych które są ruchome
- b. Sterowanie chwytakiem

- c. Obliczanie kinematyki odwrotnej
- d. Wysyłanie kątów do stawów ramienia
- e. Pobieranie pozycji efektora (do systemu wizyjnego)



Rysunek 1: Model robota w środowisku symulacyjnym

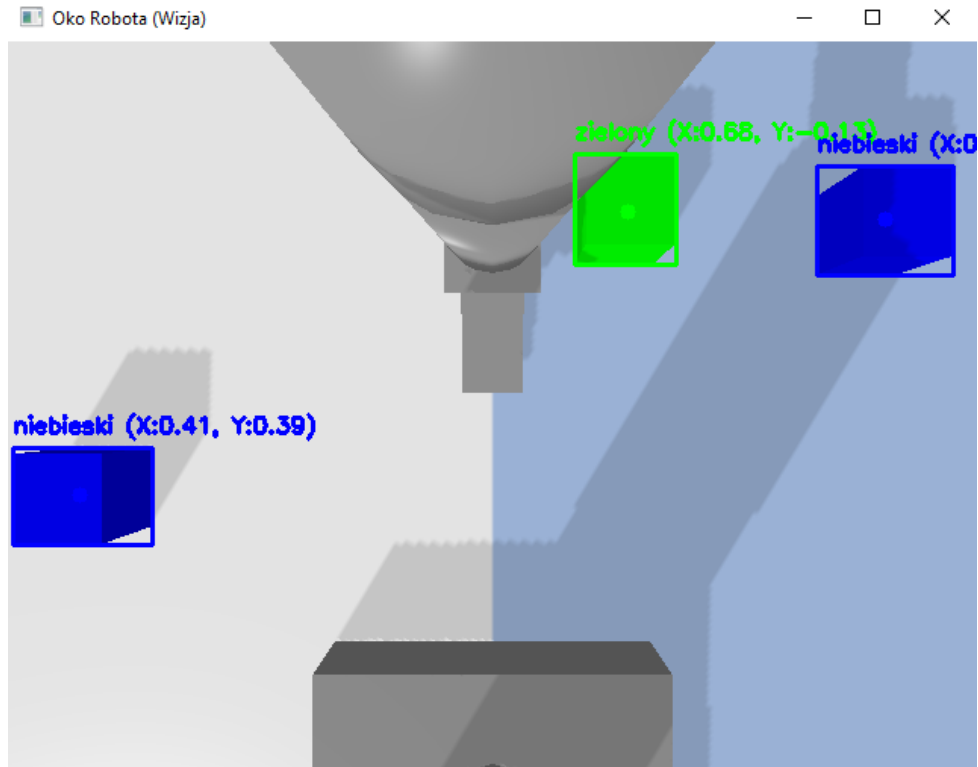
5 System wizyjny i autonomiczne sortowanie:

System wizyjny został zaimplementowany przy użyciu bibliotek numpy, opencv oraz pybullet. Działa on na zasadzie zdjęcia zrobionego przez Pybullet, przerobieniu go na format opencv (mi. RGB na BGR). Po określeniu gdzie znajdują się kolorowe kostki system przesyła ich współrzędne do pliku sterującego autonomous.py. Plik sterujący wykonuje pętlę:

- a. obserwacja
- b. dojazd nad klocek
- c. zjazd w dół
- d. chwytanie
- e. podnoszenie
- f. przeniesienie nad stos
- g. opuszczenie na stos
- h. puszczenie

- i. podnoszenie ze stosu
- j. powrót do bazy

Współrzędne stosów poszczególnych kolorów są określone sztywno w programie. Niżej załączono screenshot z pracy systemu wizyjnego.



Rysunek 2: Oko Robota (Wizja)

6 Środowisko:

Do implementacji środowiska wykorzystano bibliotekę pybullet oraz pliki urdf. Plik ze środowiskiem, poza grawitacją oraz fizyką kolizji, umożliwia tworzenie kostek do prezentacji możliwości symulacji robota.

7 Obsługa:

Program daje różne sposoby sterowania symulacją. Są to:

- Podanie współrzędnych docelowych w terminalu (Kinematyka odwrotna)
- Ustawienie położenia poszczególnych stawów z pomocą suwaków
- Autonomiczne sortowanie kostek, na stosy (Po wciśnięciu klawisza "A" aby rozpocząć sortowanie i "M" aby zakończyć)
- Sterowanie poprzez uczenie, czyli możliwość wciśnięcia klawisza "R" aby rozpocząć nagrywanie sekwencji i po zakończeniu nagrywania wciśnięcie klawisza "P", aby odtworzyć trasę.

Program również daje możliwość tworzenia kostek wciskając klawisz "C" oraz zaciskanie i puszczenie chwytaka - Spacja

8 Komentarze końcowe:

Ramię robota zaprojektowane jest jako obiekty klasy robot. Poprzez rozbicie funkcji programu na osobne pliki spełniono założenia programowania obiektowego. Był pomysł wprowadzenia dockera w celu uproszczenia obsługi programu na różnych platformach, jednak nie został on zrealizowany. Był również pomysł wprowadzenia pliku mesh zmieniającego wygląd robota z domyślnego na własne tekstury, nie został on wprowadzony.